

UNIVERSITY OF
BUCHAREST

FACULTY OF
MATHEMATICS AND
COMPUTER SCIENCE



DATA SCIENCE

Practical Machine Learning
Kaggle Competition Project Report

IMAGE NOISE DISTRIBUTION MATCHING

Student

Lungu Laura-Vanesa

Bucharest, December 2024

Contents

1 Introduction **2**

2 Exploratory Data Analysis **2**
2.1 Dataset Overview 2
2.2 First Attempts: Statistical Features 2
2.3 Why Statistical Features Don't Work 3
2.4 Exploring Frequency Domain 3

3 Feature Engineering **4**
3.1 Data Preprocessing 5
3.2 Feature Categories Overview 5
3.3 Detailed Feature Descriptions 6
3.3.1 Global Correlations 6
3.3.2 Multi-Scale Patches 7
3.3.3 Spatial Distribution 7
3.3.4 Frequency Domain 7
3.3.5 Wavelets 8
3.3.6 Statistical Moments 8
3.3.7 Rank Correlations 9
3.3.8 Overlap Patches 9
3.4 Feature Engineering Philosophy 9

4 Model Selection and Training **9**
4.1 Model Architecture Selection 9
4.2 Models Evaluated 10
4.3 Training Procedure 10
4.3.1 Data Preparation 10
4.3.2 Model Comparison 11
4.3.3 Final Training 11
4.4 Model Comparison Results 11
4.5 Hyperparameter Tuning 11
4.6 Ensemble Strategy 12

5 Results **13**

6 Conclusion **13**

1 Introduction

This project tackles a binary classification problem: given two noise images, determine whether they come from the same noise distribution or not. Each image pair gets a label - 1 if they match (same distribution), 0 if they don't. The goal is to maximize classification accuracy on a test set.

My approach had three main stages. First, I did exploratory data analysis to understand what makes matching pairs different from non-matching ones. Second, I built a feature extraction pipeline that captures how similar two images are across different aspects - spatial patterns, frequency characteristics, and statistical properties. Third, I trained gradient boosting models to learn from these features and make predictions.

2 Exploratory Data Analysis

2.1 Dataset Overview

The dataset has three parts:

- Training set: 6,078 image pairs with labels
- Validation set: 1,800 image pairs with labels
- Test set: 4,104 image pairs (no labels, for final evaluation)

Each image is 256×256 pixels, grayscale, containing noise. The training labels are balanced - roughly half are matching pairs (label=1) and half are non-matching (label=0).

2.2 First Attempts: Statistical Features

When I first looked at this problem, I figured I should start simple. For each image pair, I calculated basic statistics like mean, standard deviation, median, variance, and percentiles, then computed the absolute differences between these values for the two images. The idea was that if two images come from the same distribution, their statistics should be similar.

I tested this on a sample of 500 matching pairs and 500 non-matching pairs, and the results were surprising. For example:

- Some matching pairs had a standard deviation difference of $\Delta_{\text{std}} \approx 1.5$
- Some non-matching pairs had $\Delta_{\text{std}} \approx 0.5$

This pattern showed up across all the statistical features I tried - mean, median, percentiles, even skewness and kurtosis. There was just too much overlap between the

two classes. Matching pairs often had bigger statistical differences than non-matching pairs.

2.3 Why Statistical Features Don't Work

This observation revealed a fundamental issue with the statistical approach: the task is not about comparing the intensity distributions of two images, but rather about identifying whether they were generated by the same underlying stochastic process.

The key insight is this: two samples drawn from different noise generators can exhibit very similar statistical moments purely by chance. Conversely, two samples from the same generator will naturally differ due to the inherent randomness of the process. Therefore, comparing statistical moments provides information about those specific samples, but not necessarily about whether the generating processes are identical.

What the task requires is a method to measure pattern similarity between images - whether the structural characteristics match - rather than whether their intensity statistics are close. This observation motivated a shift toward correlation-based features that explicitly capture pairwise relationships.

2.4 Exploring Frequency Domain

Since spatial statistics weren't working, I decided to look at the frequency domain. I computed the 2D Fourier Transform (FFT) for several image pairs and visualized the magnitude spectra.

Figure 1 shows what I found: matching pairs had similar frequency patterns - the bright and dark regions in their FFT plots lined up pretty well. Non-matching pairs, on the other hand, had completely different spectral structures. The patterns just didn't match.

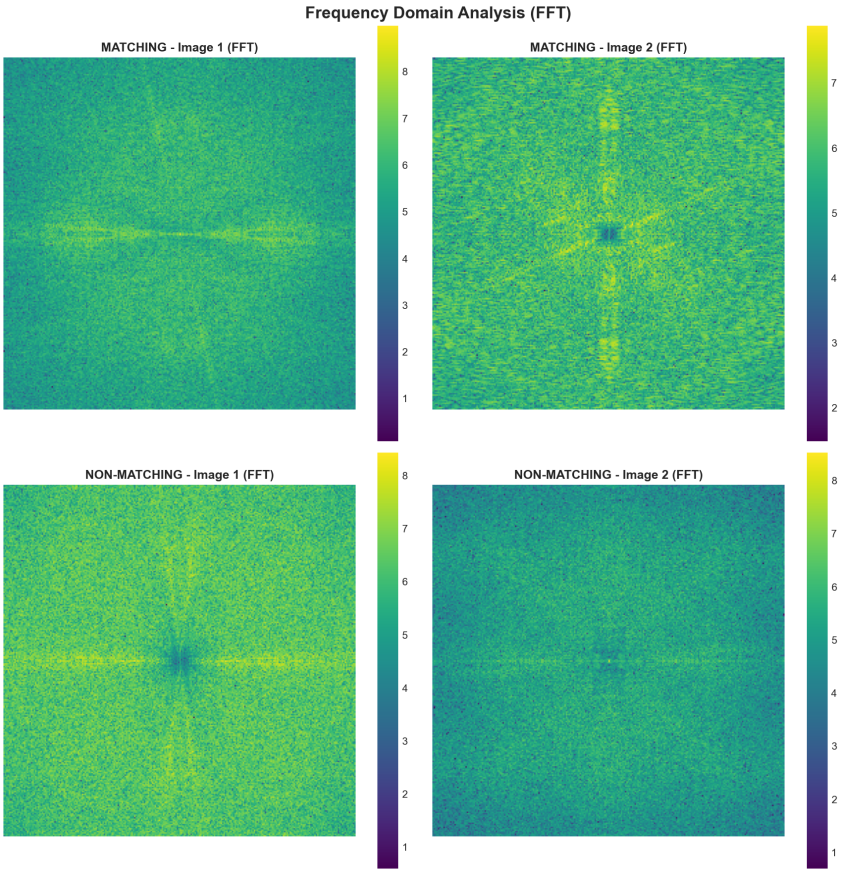


Figure 1: Frequency domain comparison. Top: matching pair with similar FFT magnitude patterns. Bottom: non-matching pair showing distinct spectral characteristics.

This was encouraging because it suggested that frequency domain features could capture structural similarities that statistical features miss. Even when two images have different spatial statistics, if they come from the same distribution, their frequency patterns should be similar.

3 Feature Engineering

After the exploratory analysis showed that statistical features alone weren't working, I needed a different approach. The key realization was that I should focus on measuring how similar two images are to each other, rather than comparing their individual properties. This led me to develop a correlation-based feature extraction pipeline.

The core idea is simple: if two images come from the same noise distribution, they should be correlated in multiple ways - spatially (do similar patterns appear in similar locations?), spectrally (do they have similar frequency content?), and structurally (do they have similar texture characteristics?). By measuring these correlations from different angles and at different scales, I could build a comprehensive picture of whether two images are related.

I ended up with 127 features organized into 8 categories. Each category captures a different aspect of similarity, and together they provide multiple independent pieces of evidence about whether two images match.

3.1 Data Preprocessing

Before extracting features, I applied some basic preprocessing to ensure numerical stability:

1. **Type conversion:** Convert images to `float64` (prevents overflow in calculations)
2. **Clean invalid values:** Replace NaN and infinite values with zeros
3. **Verify dimensions:** Check that images are 256×256 pixels
4. **Safe correlation:** Use a wrapper function that handles edge cases (zero variance, mismatched lengths) and returns 0.0 when correlation is undefined

These steps ensure that all the feature computations run smoothly without numerical errors.

3.2 Feature Categories Overview

The 127 features are organized into 8 categories based on what aspect of similarity they measure. Table 1 provides an overview.

Table 1: Feature architecture organized by similarity aspect

Category	What It Measures
Global correlations	Overall similarity using different correlation metrics (Pearson, Spearman, normalized, etc.)
Multi-scale patches	Local pattern similarity at three resolutions: coarse (128×128), medium (64×64), and fine (32×32)
Spatial distribution	Whether similarity is uniform across the image or varies between center and edges
Frequency domain	Spectral characteristics using FFT (power, phase) and DCT (energy distribution)
Wavelets	Multi-resolution patterns that capture both spatial location and frequency content
Statistical moments	Basic distribution differences (mean, std, percentiles) for complementary information
Rank correlations	Ordinal relationships that are robust to intensity transformations
Overlap patches	Sliding window analysis to capture patterns that might fall on grid boundaries

3.3 Detailed Feature Descriptions

3.3.1 Global Correlations

The most straightforward way to measure similarity is to compute the correlation between the two images when treated as long vectors (flattening the 256×256 pixels into 65,536 values).

However, a single correlation metric can miss important patterns. For example, if one image has a few extreme outliers, Pearson correlation might be low even if the overall pattern is similar. To handle this, I computed multiple types of correlations:

Basic correlations:

- Pearson: standard linear correlation
- Spearman: rank-based correlation (robust to outliers)
- Absolute correlation: correlation of magnitudes (ignores sign)
- Squared correlation: emphasizes high-intensity regions
- Sign correlation: only looks at positive/negative pattern

Stratified and transformed correlations:

- Quantile-based: separate correlations for low (0-25%), mid (25-75%), and high (75-100%) intensity ranges
- Centered: correlation after removing median (removes DC offset)
- Normalized: correlation after min-max scaling (scale-invariant)
- Histogram: correlation of intensity distributions rather than spatial patterns

The rationale is that if one type of correlation fails (due to outliers, non-linearity, or scale differences), other types might still capture the underlying similarity.

3.3.2 Multi-Scale Patches

Rather than only looking at global correlations, I also examined whether similar patterns appear in corresponding local regions. I did this at three different scales:

Coarse scale (2×2 grid, 128×128 patches): Divides each image into 4 large quadrants. For each quadrant pair, I computed the correlation, mean difference, and max difference. This captures whether the global structure matches.

Fine scale (8×8 grid, 32×32 patches): I sampled 24 strategic positions (corners, edges, center, transitions) rather than all 64 patches. This resolution is fine enough to capture local texture patterns but not so fine that it overfits to individual pixel noise.

Intermediate scale (4×4 grid, 64×64 patches): Complete coverage at medium resolution. This bridges the gap between coarse and fine scales.

The idea is that noise patterns can appear at different spatial frequencies. By analyzing at multiple scales, I ensure that I capture patterns regardless of their size.

3.3.3 Spatial Distribution

These features check whether similarity is uniform across the image or concentrated in specific regions.

Center vs edges: I computed separate correlations for the central 128×128 region and for the four 64×64 corners. If the center correlation is high but corner correlations are low (or vice versa), it suggests that the noise process has spatial variation or boundary effects.

Diagonal patches: Four patches along the main diagonal (top-left to bottom-right) capture whether there's a consistent gradient pattern in both images.

3.3.4 Frequency Domain

Spatial features alone might miss patterns that become clear in the frequency domain. I used two frequency representations:

FFT (Fast Fourier Transform): The FFT decomposes the image into frequency components. I extracted:

- Power spectrum correlation: do the images have similar frequency content?
- Phase correlation: do they have similar spatial structure?
- Low vs high frequency correlations: some noise types differ mainly in specific frequency ranges
- Directional correlations: horizontal vs vertical frequency patterns

DCT (Discrete Cosine Transform): Similar to FFT but uses only real-valued cosine functions. DCT concentrates most energy in low-frequency coefficients, making it particularly good at detecting low-frequency pattern similarity.

The reason for including both is that different noise types have characteristic spectral signatures. For instance, white noise has a flat spectrum, while colored noise has frequency-dependent power. Two images from the same distribution should have matching spectral patterns.

3.3.5 Wavelets

Wavelets are a hybrid between spatial and frequency analysis - they tell you what frequencies are present and where they occur spatially. This is powerful because two images might have the same frequency content overall but distributed differently in space.

I used Haar wavelets to decompose each image into:

- Approximation coefficients (cA): low-frequency content
- Detail coefficients (cH, cV, cD): high-frequency content in horizontal, vertical, and diagonal directions

For each component, I computed correlations and energy ratios between the two images. This captures whether local texture patterns match.

3.3.6 Statistical Moments

Even though the EDA showed that statistical features alone don't work well, I included them anyway for two reasons. First, they might provide useful complementary information when combined with strong correlation features. Second, gradient boosting models can learn to use them conditionally - for example, "if correlation is high, also check if the means are close."

The features include differences in mean, standard deviation, median, skewness, kurtosis, percentiles (25th, 50th, 75th), minimum, and maximum.

3.3.7 Rank Correlations

These features measure whether the two images preserve ordinal relationships - do pixels that are bright in one image tend to be bright in the other, regardless of the exact values?

Rank correlations are useful because they're invariant to monotonic transformations. If one distribution is a logarithmic or exponential transformation of another, the pixel values differ but the ranks stay the same. I computed global rank correlation, rank difference statistics, and rank correlations for specific regions (center, high-intensity areas).

3.3.8 Overlap Patches

All the previous patch-based features used non-overlapping grids (2×2, 4×4, 8×8). However, this approach can miss patterns that happen to fall on grid boundaries. To solve this, I added sliding window features with 50% overlap.

I used 64×64 windows with a stride of 32 pixels, sampling 7 strategic positions (corners, center, offsets). This ensures complete spatial coverage without boundary gaps.

3.4 Feature Engineering Philosophy

The overall design follows three principles:

1. Multiple representations: By combining spatial, frequency, and wavelet features, I capture complementary information. Patterns that are invisible in one domain might be obvious in another.

2. Robustness through diversity: Using multiple correlation types (Pearson, Spearman, normalized, etc.) ensures that at least some features remain informative even when certain metrics fail due to outliers or non-linearity.

3. Multi-scale analysis: Noise patterns can appear at different spatial resolutions. Analyzing at coarse, medium, and fine scales ensures I don't miss patterns regardless of their size.

This comprehensive feature set provides 127 independent measurements of similarity, giving the gradient boosting models rich information to learn from.

4 Model Selection and Training

4.1 Model Architecture Selection

Based on the correlation-based nature of the features, gradient boosting models were selected as the primary approach. Three characteristics make gradient boosting particularly suitable for this task:

Sequential learning: Boosting builds trees sequentially, with each tree correcting errors from previous trees. This iterative refinement is effective for learning subtle correlation patterns.

Feature interaction discovery: Tree-based models automatically find which combinations of features are useful. This is necessary because checking just one feature at a time doesn't work well - two images might have high correlation overall but completely different frequency characteristics. The model learns to look at multiple aspects together before deciding if images match.

Robustness to irrelevant features: Since I extracted 127 features and wasn't sure which ones would be most useful, gradient boosting was helpful because it automatically figures out which features matter and ignores the rest. This meant I could experiment with many different feature types without worrying too much about adding noise.

4.2 Models Evaluated

I tested six different model configurations on the validation set:

LightGBM (2 configurations): Fast training with leaf-wise tree growth.

- Baseline: 1200 trees, 255 leaves, learning rate 0.015
- Deep: 1500 trees, 300 leaves, learning rate 0.012

XGBoost (2 configurations): Stable level-wise tree growth.

- Baseline: 1200 trees, depth 12, learning rate 0.015
- Deep: 1500 trees, depth 15, learning rate 0.012

RandomForest: 500 trees, depth 20 (baseline for comparison)
GradientBoosting: Sklearn implementation, 500 trees (baseline for comparison)

4.3 Training Procedure

4.3.1 Data Preparation

All 127 features were standardized using sklearn's StandardScaler:

$$X_{\text{scaled}} = \frac{X - \mu}{\sigma} \tag{1}$$

This ensures features with different ranges (correlations in [-1,1] vs pixel differences in [0,255]) contribute equally. Mean and standard deviation were computed on the training set and applied consistently to validation and test sets.

4.3.2 Model Comparison

Six models were trained on the training set (6,078 samples) and evaluated on the validation set (1,800 samples) using five metrics:

- **Accuracy:** Primary competition metric
- **AUC:** Ranking quality across thresholds
- **F1 score:** Balance between precision and recall
- **Spearman/Kendall:** Probability calibration quality

4.3.3 Final Training

After identifying the best models from validation results, I retrained them on the combined train with validation dataset (7,878 samples total). Since the competition test set is completely separate and never seen during training, using all available labeled data was safe and maximized training data.

4.4 Model Comparison Results

Table 2 shows validation set performance for all tested models.

Table 2: Validation set performance comparison (1,800 samples)

Model	Accuracy	AUC	F1	Spearman	Kendall
LightGBM_baseline	**83.06%**	0.9143	0.8272	0.7174	0.5860
XGBoost_baseline	83.00%	0.9155	0.8275	0.7195	0.5876
XGBoost_deep	82.78%	**0.9159**	0.8258	**0.7202**	**0.5882**
LightGBM_deep	82.28%	0.9118	0.8197	0.7132	0.5825
GradientBoosting	81.22%	0.9011	0.8080	0.6947	0.5673
RandomForest	78.89%	0.8667	0.7793	0.6351	0.5187

Key findings: All gradient boosting models achieved 82-83% accuracy, significantly outperforming RandomForest (78.89%) and sklearn's GradientBoosting (81.22%). The top four models differed by less than 1%, suggesting that feature quality mattered more than model architecture. LightGBM_baseline achieved the best validation accuracy while maintaining strong correlation metrics, making it the primary candidate for further optimization.

4.5 Hyperparameter Tuning

Since LightGBM_baseline showed the best overall performance, I systematically tuned its hyperparameters to find the optimal configuration. Table 3 presents the results.

Table 3: LightGBM hyperparameter tuning results

Hyperparameter	Value	Accuracy	Spearman	Finding
Estimators	800	82.28%	0.7202	Underfitting
	1200	82.78%	0.7242	Good balance
	1500	82.83%	0.7239	Diminishing returns
Learning rate	0.01	82.72%	0.7233	Too slow
	0.015	82.78%	0.7242	Baseline
	0.02	**83.33%**	0.7262	Best accuracy
Tree complexity	127 leaves	**83.00%**	0.7231	Best generalization
	255 leaves	82.78%	0.7242	Baseline
	511 leaves	82.56%	0.7208	Overfitting
Regularization	0.0 (weak)	82.78%	0.7242	Baseline
	0.2 (medium)	82.89%	0.7209	Optimal
	0.5 (strong)	82.83%	0.7190	Too restrictive

Key insights: Increasing trees from 800 to 1200 improved accuracy by 0.5%, but 1200 to 1500 only added 0.05% (diminishing returns). Surprisingly, learning rate 0.02 achieved the highest accuracy (83.33%), though the Spearman improvement was minimal. Simpler trees (127 leaves) outperformed complex ones (511 leaves showed overfitting). Moderate regularization (0.2) balanced performance and generalization.

Final configuration: Despite 0.02 learning rate achieving slightly higher accuracy, I chose the baseline configuration for consistency:

- 1200 estimators (balance of accuracy and speed)
- Learning rate 0.015 (more stable across runs)
- 255 leaves (consistent performance)
- Feature fraction 0.7, bagging fraction 0.7, L1/L2 regularization 0.2

4.6 Ensemble Strategy

Rather than relying on a single model, I combined predictions from LightGBM and XGBoost:

$$P_{\text{ensemble}} = 0.6 \cdot P_{\text{LightGBM}} + 0.4 \cdot P_{\text{XGBoost}} \tag{2}$$

LightGBM received higher weight (60%) due to better validation accuracy. The two models use different tree-growing strategies (leaf-wise vs level-wise), making their errors somewhat independent: when one makes a mistake, the other might be correct, and averaging reduces overall errors.

5 Results

The final model achieved 90.39% accuracy on the competition test set. The substantial improvement from validation (83.06%) to test (90.39%) suggests the validation set was more challenging, though this gap also indicates that the correlation-based features captured patterns that generalized exceptionally well to unseen data.

Three models were submitted to the competition:

- **LightGBM:** 90.39% (best performance)
- **XGBoost:** 89.80%
- **Ensemble:** 90.18%

Interestingly, LightGBM alone outperformed the ensemble, suggesting that for this particular test set, LightGBM's predictions were highly accurate and ensembling introduced slight noise. However, having multiple submissions provided valuable risk mitigation.

6 Conclusion

This work showed that simple statistical features are insufficient for distinguishing noise images generated by the same distribution. A correlation-based, multi-domain feature representation capturing spatial and frequency similarities proved significantly more effective. Combined with gradient boosting models, this approach achieved 90.39% accuracy on the test set, highlighting the importance of feature design over model complexity for this task.